

Universidade Estácio de Sá

Curso: Desenvolvimento Full Stack

Disciplina: DGT2811 – Desenvolvimento Back-End Corporativo com Java e Cloud

Matrícula: 2024 0446 6396

Semestre Letivo: 2026.3

Aluno: Pedro Henrique Oliveira Remigio

Título da Prática

Implementação de Sistema Cadastral Web com Servlets, JPA e JEE

Objetivo da Prática

Implementar um sistema cadastral Web completo utilizando as tecnologias da plataforma Java EE, incluindo:

Persistência de dados com JPA (Java Persistence API)

Regras de negócio com EJBs (Enterprise Java Beans)

Interface Web com Servlets e JSPs

Melhoria de design com o framework Bootstrap

Códigos Implementados

persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<persistence version="3.2"
```

```
  xmlns="https://jakarta.ee/xml/ns/persistence"
```

```
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
  xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
```

```
    https://jakarta.ee/xml/ns/persistence/persistence_3_2.xsd">
```

```

<persistence-unit name="CadastroEE-ejbPU" transaction-type="JTA">
    <jta-data-source>jdbc/loja</jta-data-source>
    <exclude-unlisted-classes>>false</exclude-unlisted-classes>
    <properties/>
</persistence-unit>
</persistence>

```

Produto.java (Entidade JPA)

```
package cadastroee.model;
```

```
import jakarta.persistence.*;
```

```
import java.io.Serializable;
```

```
@Entity
```

```
@Table(name = "Produto")
```

```
@NamedQueries({
```

```
    @NamedQuery(name = "Produto.findAll", query = "SELECT p FROM Produto p"),
```

```
    @NamedQuery(name = "Produto.findById", query = "SELECT p FROM Produto p
WHERE p.id = :id"),
```

```
    @NamedQuery(name = "Produto.findByName", query = "SELECT p FROM
Produto p WHERE p.nome = :nome"),
```

```
    @NamedQuery(name = "Produto.findByQuantidade", query = "SELECT p FROM
Produto p WHERE p.quantidade = :quantidade"),
```

```
    @NamedQuery(name = "Produto.findByPrecoVenda", query = "SELECT p FROM
Produto p WHERE p.precoVenda = :precoVenda")
```

```
})
```

```
public class Produto implements Serializable {
```

```
    private static final long serialVersionUID = 1L;
```

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

@Basic(optional = false)

@Column(name = "id")

private Integer id;

@Column(name = "nome")

private String nome;

@Column(name = "quantidade")

private Integer quantidade;

@Column(name = "precoVenda")

private Float precoVenda;

public Produto() {}

public Produto(Integer id) { this.id = id; }

// Getters e Setters

public Integer getId() { return id; }

public void setId(Integer id) { this.id = id; }

public String getNome() { return nome; }

public void setNome(String nome) { this.nome = nome; }

public Integer getQuantidade() { return quantidade; }

public void setQuantidade(Integer quantidade) { this.quantidade = quantidade; }

public Float getPrecoVenda() { return precoVenda; }

public void setPrecoVenda(Float precoVenda) { this.precoVenda = precoVenda; }

}

AbstractFacade.java

```
package cadastroee.controller;
```

```
import jakarta.persistence.EntityManager;
```

```
import java.util.List;
```

```
public abstract class AbstractFacade<T> {
```

```
    private Class<T> entityClass;
```

```
    public AbstractFacade(Class<T> entityClass) { this.entityClass = entityClass; }
```

```
    protected abstract EntityManager getEntityManager();
```

```
    public void create(T entity) { getEntityManager().persist(entity); }
```

```
    public void edit(T entity) { getEntityManager().merge(entity); }
```

```
    public void remove(T entity) {  
getEntityManager().remove(getEntityManager().merge(entity)); }
```

```
    public T find(Object id) { return getEntityManager().find(entityClass, id); }
```

```
    public List<T> findAll() {
```

```
        jakarta.persistence.criteria.CriteriaQuery cq =  
getEntityManager().getCriteriaBuilder().createQuery();
```

```
        cq.select(cq.from(entityClass));
```

```
        return getEntityManager().createQuery(cq).getResultList();
```

```
    }
```

```
}
```

ProdutoFacadeLocal.java (Interface EJB)

```
package cadastroee.controller;
```

```
import cadastroee.model.Produto;
```

```
import jakarta.ejb.Local;
```

```
import java.util.List;
```

```
@Local
```

```
public interface ProdutoFacadeLocal {
```

```
    void create(Produto produto);
```

```
    void edit(Produto produto);
```

```
    void remove(Produto produto);
```

```
    Produto find(Object id);
```

```
    List<Produto> findAll();
```

```
}
```

```
ProdutoFacade.java (Session Bean)
```

```
package cadastroee.controller;
```

```
import cadastroee.model.Produto;
```

```
import jakarta.ejb.Stateless;
```

```
import jakarta.persistence.EntityManager;
```

```
import jakarta.persistence.PersistenceContext;
```

```
@Stateless
```

```
public class ProdutoFacade extends AbstractFacade<Produto> implements  
ProdutoFacadeLocal {
```

```
    @PersistenceContext(unitName = "CadastroEE-ejbPU")
```

```
    private EntityManager em;
```

@Override

protected EntityManager getEntityManager() { return em; }

public ProdutoFacade() { super(Produto.class); }

}

ServletProduto.java

package cadastroee.servlets;

import cadastroee.controller.ProdutoFacadeLocal;

import cadastroee.model.Produto;

import jakarta.ejb.EJB;

import jakarta.servlet.*;

import jakarta.servlet.http.*;

import java.io.*;

import java.util.List;

public class ServletProduto extends HttpServlet {

@EJB

ProdutoFacadeLocal facade;

protected void processRequest(HttpServletRequest request,
HttpServletResponse response)

throws ServletException, IOException {

response.setContentType("text/html;charset=UTF-8");

try (PrintWriter out = response.getWriter()) {

```

        out.println("<!DOCTYPE html><html><head><title>Lista de
Produtos</title></head><body>");

        out.println("<h1>Lista de Produtos</h1><ul>");

        List<Produto> produtos = facade.findAll();

        for (Produto p : produtos) {

            out.println("<li>" + p.getId() + " - " + p.getNome()

                + " | Qtd: " + p.getQuantidade()

                + " | Preço: R$ " + String.format("%.2f", p.getPrecoVenda()) + "</li>");

        }

        out.println("</ul></body></html>");

    }

}

```

@Override

```

protected void doGet(HttpServletRequest req, HttpServletResponse res)

    throws ServletException, IOException { processRequest(req, res); }

```

@Override

```

protected void doPost(HttpServletRequest req, HttpServletResponse res)

    throws ServletException, IOException { processRequest(req, res); }

}

```

ServletProdutoFC.java (Front Controller)

```

package cadastroee.servlets;

import cadastroee.controller.ProdutoFacadeLocal;
import cadastroee.model.Produto;
import jakarta.ejb.EJB;
import jakarta.servlet.*;

```

```
import jakarta.servlet.http.*;

import java.io.IOException;

import java.util.List;


public class ServletProdutoFC extends HttpServlet {

    @EJB

    ProdutoFacadeLocal facade;


    protected void processRequest(HttpServletRequest request,
    HttpServletResponse response)

        throws ServletException, IOException {


        String acao = request.getParameter("acao");

        String destino = ("formIncluir".equals(acao) || "formAlterar".equals(acao))

            ? "ProdutoDados.jsp" : "ProdutoLista.jsp";


        if ("listar".equals(acao)) {

            request.setAttribute("lista", facade.findAll());

        } else if ("formAlterar".equals(acao)) {

            int id = Integer.parseInt(request.getParameter("id"));

            request.setAttribute("produto", facade.find(id));

        } else if ("excluir".equals(acao)) {

            int id = Integer.parseInt(request.getParameter("id"));

            facade.remove(facade.find(id));

            request.setAttribute("lista", facade.findAll());

        } else if ("alterar".equals(acao)) {

            int id = Integer.parseInt(request.getParameter("id"));
```



```

        Produto p = facade.find(id);

        p.setNome(request.getParameter("nome"));

        p.setQuantidade(Integer.parseInt(request.getParameter("quantidade")));

        p.setPrecoVenda(Float.parseFloat(request.getParameter("precoVenda")));

        facade.edit(p);

        request.setAttribute("lista", facade.findAll());

    } else if ("incluir".equals(acao)) {

        Produto p = new Produto();

        p.setNome(request.getParameter("nome"));

        p.setQuantidade(Integer.parseInt(request.getParameter("quantidade")));

        p.setPrecoVenda(Float.parseFloat(request.getParameter("precoVenda")));

        facade.create(p);

        request.setAttribute("lista", facade.findAll());

    }

    request.getRequestDispatcher(destino).forward(request, response);
}

```

@Override

```

protected void doGet(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException { processRequest(req, res); }

```

@Override

```

protected void doPost(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException { processRequest(req, res); }

```

}

web.xml

<?xml version="1.0" encoding="UTF-8"?>

```
<web-app version="6.0" xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
    https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd">
  <servlet>
    <servlet-name>ServletProduto</servlet-name>
    <servlet-class>cadastroee.servlets.ServletProduto</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>ServletProduto</servlet-name>
    <url-pattern>/ServletProduto</url-pattern>
  </servlet-mapping>
  <servlet>
    <servlet-name>ServletProdutoFC</servlet-name>
    <servlet-class>cadastroee.servlets.ServletProdutoFC</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>ServletProdutoFC</servlet-name>
    <url-pattern>/ServletProdutoFC</url-pattern>
  </servlet-mapping>
  <session-config>
    <session-timeout>30</session-timeout>
  </session-config>
</web-app>
```

ProdutoLista.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
```

```
<%@page import="cadastroee.model.Produto"%>
```

```

<%@page import="java.util.List"%>

<!DOCTYPE html>

<html>

<head>

<meta charset="UTF-8">

<title>Lista de Produtos</title>

<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

</head>

<body class="container">

<h2 class="mt-4 mb-3">Lista de Produtos</h2>

<a href="ServletProdutoFC?acao=formIncluir" class="btn btn-primary m-2">Novo Produto</a>

<table class="table table-striped mt-3">

<thead class="table-dark">

<tr><th>ID</th><th>Nome</th><th>Quantidade</th><th>Preço de
Venda</th><th>Ações</th></tr>

</thead>

<tbody>

<% List<Produto> lista = (List<Produto>) request.getAttribute("lista");
if (lista != null) { for (Produto p : lista) { %>

<tr>

<td><%= p.getId() %></td>

<td><%= p.getNome() %></td>

<td><%= p.getQuantidade() %></td>

<td>R$ <%= String.format("%.2f", p.getPrecoVenda()) %></td>

```

```

        <td>

            <a href="ServletProdutoFC?acao=formAlterar&id=<%= p.getId() %>"
class="btn btn-primary btn-sm">Alterar</a>

            <a href="ServletProdutoFC?acao=excluir&id=<%= p.getId() %>" class="btn
btn-danger btn-sm">Excluir</a>

        </td>

    </tr>

    <% } } %>

</tbody>

</table>

</body>

</html>

```

ProdutoDados.jsp

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>

<%@page import="cadastroee.model.Produto"%>

<!DOCTYPE html>

<html>

    <head>

        <meta charset="UTF-8">

        <title>Cadastro de Produto</title>

        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">

        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.j
s"></script>

    </head>

    <body class="container">

        <% Produto produto = (Produto) request.getAttribute("produto");

```

```

String acao = (produto == null) ? "incluir" : "alterar";

String titulo = (produto == null) ? "Incluir Produto" : "Alterar Produto";

String btnTexto = (produto == null) ? "Incluir" : "Salvar Alterações"; %>

<h2 class="mt-4 mb-3"><%= titulo %></h2>

<form action="ServletProdutoFC" method="post" class="form">

    <input type="hidden" name="acao" value="<%= acao %>">

    <% if ("alterar".equals(acao)) { %>

        <input type="hidden" name="id" value="<%= produto.getId() %>">

    <% } %>

    <div class="mb-3">

        <label class="form-label">Nome:</label>

        <input type="text" name="nome" class="form-control"

            value="<%= (produto != null) ? produto.getNome() : "" %>">

    </div>

    <div class="mb-3">

        <label class="form-label">Quantidade:</label>

        <input type="number" name="quantidade" class="form-control"

            value="<%= (produto != null) ? produto.getQuantidade() : "" %>">

    </div>

    <div class="mb-3">

        <label class="form-label">Preço de Venda:</label>

        <input type="text" name="precoVenda" class="form-control"

            value="<%= (produto != null) ? String.format("%.2f",
produto.getPrecoVenda()) : "" %>">

    </div>

    <button type="submit" class="btn btn-primary"><%= btnTexto %></button>

    <a href="ServletProdutoFC?acao=listar" class="btn btn-secondary ms-
2">Cancelar</a>

</form>

```

</body>

</html>

Resultados da Execução

O sistema foi executado com sucesso no servidor Eclipse GlassFish 8.0.0 com banco de dados SQL Server 2025 Express. As funcionalidades testadas foram:

Listagem de produtos acessando <http://localhost:8080/CadastroEE-war/ServletProdutoFC?acao=listar> :exibiu corretamente todos os produtos cadastrados em tabela formatada com Bootstrap.

Inclusão de produto: o formulário foi preenchido e o novo produto apareceu na listagem após a confirmação.

Alteração de produto: o formulário foi pré-preenchido com os dados existentes e as alterações foram persistidas.

Exclusão de produto: o produto foi removido e a listagem atualizada automaticamente.

ServletProduto acessando <http://localhost:8080/CadastroEE-war/ServletProduto> :listagem simples em HTML puro funcionou corretamente.

Análise e Conclusão

1º Procedimento: Camadas de Persistência e Controle

Como é organizado um projeto corporativo no NetBeans?

Um projeto Enterprise Application no NetBeans é organizado em três módulos distintos: o projeto principal (EAR), que empacota os demais; o módulo EJB (CadastroEE-ejb), responsável pelas entidades JPA e pelos Session Beans; e o módulo WAR (CadastroEE-war), responsável pela camada Web com Servlets e JSPs. Essa estrutura reflete a arquitetura em camadas da plataforma Jakarta EE, separando claramente a persistência, as regras de negócio e a apresentação.

Qual o papel das tecnologias JPA e EJB na construção de um aplicativo Web Java?

O JPA (Jakarta Persistence API) é responsável pelo mapeamento objeto-relacional, permitindo que classes Java (entidades) representem tabelas do banco de dados sem a necessidade de escrever SQL manualmente. Os EJBs (Enterprise Java Beans), especificamente os Session Beans Stateless, encapsulam as regras de negócio e gerenciam transações automaticamente via contêiner Jakarta EE,

garantindo segurança, escalabilidade e rastreabilidade das operações sobre o banco.

Como o NetBeans viabiliza a melhoria de produtividade ao lidar com JPA e EJB?

O NetBeans oferece assistentes (wizards) que automatizam a geração de entidades JPA a partir de tabelas existentes no banco (New Entity Classes from Database) e de Session Beans para cada entidade (New Session Beans for Entity Classes), eliminando a necessidade de escrever manualmente código repetitivo de CRUD. Isso permite ao desenvolvedor focar na lógica de negócio.

O que são Servlets e como o NetBeans suporta sua construção?

Servlets são componentes Java que processam requisições HTTP no lado do servidor, funcionando como controladores na arquitetura MVC. O NetBeans oferece um assistente de criação de Servlet que gera automaticamente a classe com os métodos doGet, doPost e processRequest, além de registrá-la no web.xml, acelerando o desenvolvimento.

Como é feita a comunicação entre Servlets e Session Beans?

A comunicação é feita por injeção de dependência utilizando a anotação @EJB no Servlet. O contêiner Jakarta EE é responsável por localizar e injetar a implementação correta do Session Bean (via interface local anotada com @Local), sem que o desenvolvedor precise instanciar o objeto manualmente. Isso garante baixo acoplamento e facilita a manutenção.

2º Procedimento: Interface Cadastral com Servlet e JSPs

Como funciona o padrão Front Controller na arquitetura MVC?

O padrão Front Controller centraliza o recebimento de todas as requisições em um único Servlet (ServletProdutoFC), que analisa o parâmetro acao e decide qual operação executar e para qual JSP redirecionar. Isso organiza o fluxo da aplicação, evitando lógica espalhada em múltiplos pontos e facilitando a manutenção. Na arquitetura MVC: o Servlet é o Controller, as entidades JPA são o Model, e os JSPs são a View.

Quais as diferenças e semelhanças entre Servlets e JSPs?

Ambos processam requisições HTTP e geram respostas para o cliente. A diferença é que Servlets são classes Java puras, mais adequadas para lógica de controle, enquanto JSPs são páginas com HTML mesclado a código Java (scriptlets), mais adequadas para apresentação. Internamente, o servidor converte cada JSP em um Servlet antes de executá-lo.

Qual a diferença entre redirecionamento simples e o método forward do RequestDispatcher?

O redirecionamento (`response.sendRedirect`) envia uma nova requisição ao cliente, perdendo os atributos da requisição original. O forward do RequestDispatcher transfere a requisição internamente no servidor para outro recurso (JSP ou Servlet), mantendo todos os atributos adicionados com `request.setAttribute()`. Parâmetros (`request.getParameter`) são dados enviados pelo cliente na URL ou formulário; atributos (`request.getAttribute`) são dados definidos pelo servidor para comunicação interna entre componentes.

3º Procedimento: Melhorando o Design com Bootstrap

Como o framework Bootstrap é utilizado?

O Bootstrap é incluído via CDN nas páginas JSP, adicionando seu CSS e JavaScript no `<head>`. As classes do Bootstrap são aplicadas diretamente nos elementos HTML: como container no `<body>`, `table table-striped` nas tabelas, `btn btn-primary` nos botões: sem necessidade de escrever CSS personalizado.

Por que o Bootstrap garante a independência estrutural do HTML?

O Bootstrap aplica estilos através de classes CSS sem alterar a estrutura semântica do HTML. O mesmo HTML permanece válido e acessível independentemente do framework visual utilizado, pois a separação entre estrutura (HTML) e apresentação (CSS) é mantida. Isso facilita futuras trocas de framework ou personalizações de estilo.

Qual a relação entre Bootstrap e a responsividade da página?

O Bootstrap é construído sobre um sistema de grid responsivo que adapta automaticamente o layout a diferentes tamanhos de tela (celular, tablet, desktop) usando media queries CSS. Componentes como container já possuem margens e

larguras máximas que se ajustam ao viewport, garantindo que a aplicação seja utilizável em qualquer dispositivo sem configuração adicional.

Repositório GitHub: <https://github.com/pedro-remigio/M3NV01>